

ATTORNEY SHIELD

ASI 2.0 - Native Android & iOS

Attorney Shield 2.0 - Backend Requirements

What the native apps need from the backend

2026-08-12

Colour system: Attorney-Shield Color Scheme Review

Repository: samson-phillip/asi-claude

DRAFT FOR APPROVAL

Contents

Summary	3
1. Blocking us today	3
1.1 The GraphQL gateway URL - BLOCKING	3
1.2 A dev member account - BLOCKING	3
1.3 A window with a dev attorney online - HIGH	4
2. Questions that change how we build	4
2.1 Is there a token refresh? - HIGH	4
2.2 Is there a one-time-code sign-in? - HIGH	4
2.3 What is the deep-link URL, exactly? - HIGH	4
2.4 Domain verification files - HIGH (web task, not backend)	5
3. Endpoints we need for the rest of the app	5
3.1 Native registration - screens 08-12 - BLOCKING Phase 3	5
3.2 Situation preferences - screens 13B, 13C, 27B - HIGH	6
3.3 Document vault / "Digital Glovebox" - screens 14, 14A-14D, 30A, 31 - HIGH	7
3.4 Activity timeline - screen 32 - LATER	7
3.5 Plan, payment method, family sub-accounts - screens 33A, 33B, 33D - LATER	7
3.6 Notifications and the nudge system - screens 15, 22-26 - LATER	8
3.7 Trial and guest - screens V1-V2, T1-T8, G1-G3 - LATER	8
4. Conventions we have matched	9
5. One security note	9
6. What we will do as each lands	9
Contact	10

From: mobile (Android + iOS) **Date:** 2026-08-12 **Status:** the app spine is built and working. Everything below is what we need from the backend to go further.

Priority key: BLOCKING = we are stopped now · HIGH = blocks a phase · LATER = needed, not urgent.

Summary

The native apps are built as far as the current API allows: welcome carousel, sign-in, home, incident selection, the video call flow, and the web->app deep link. That works today and is tested.

Roughly two-thirds of the designed product has no backend at all. Native registration, the document vault, family sub-accounts, activity, notifications, and the trial and guest flows have no endpoints, so they cannot be built as functioning features.

There are also **two small things blocking us right now** that are much cheaper than anything else on this page.

We are not asking for payment or checkout - we can see those live on the web, and the app correctly hands off and picks up afterwards.

1. Blocking us today

These two are the difference between "the app works end to end" and "the app cannot sign anyone in".

1.1 The GraphQL gateway URL - BLOCKING

`member-client` is served same-origin behind a proxy, so its config is just `GRAPHQL_URL = "/query"`. The real host is not written down anywhere in the repo. A native app has no origin to be relative to.

We need: the absolute gateway URL per environment.

Environment	URL
dev	?
staging	?
production	?

We already have the comms service - `https://comms-dev.attorneyshield.io` - from `serve-proxy.mjs`, and it responds correctly.

1.2 A dev member account - BLOCKING

We have the seeded org, jurisdiction and queue IDs from `config.ts`, but no account to sign in with.

We need: an email and password on `ias_dev`.

1.3 A window with a dev attorney online - HIGH

POST /api/vonage/video/member-call currently returns:

```
409 no attorney is available to take this call
```

That is correct behaviour and it confirmed the contract works. But it means **we have never completed a real video call**. We have proven the Vonage SDK loads, reaches Vonage's servers, and reports back correctly on both platforms - the only unproven step is a live session actually connecting.

We need: a scheduled window with an attorney available on dev, or a way to seed one. This is the last thing we would want unverified before a release.

2. Questions that change how we build

Not blocking, but each one changes a design decision we have already had to make.

2.1 Is there a token refresh? - HIGH

login returns a refreshToken, but member-client never sends it and we can find no refresh operation.

Why it matters more for us than for web: a browser tab is short-lived; a native app sits backgrounded for days. Without refresh, the only correct behaviour is to end the session and send the member back to sign-in - and this is an app people open *during a police stop*. That is the worst possible moment for it.

Currently implemented: a 401/403 clears the session and shows "Your session has expired. Please sign in again."

We need to know: does a refresh mutation exist? If so, its shape and the access-token lifetime. If not, is one planned?

2.2 Is there a one-time-code sign-in? - HIGH

Design reference screen 13A says members can "*sign in with a one-time text code instead*" of a password. We can find no OTP endpoint.

Currently implemented: email + password only.

We need to know: does a request-code / verify-code pair exist or is it planned? (See §3.1 - it is likely the same mechanism as phone verification.)

2.3 What is the deep-link URL, exactly? - HIGH

Screens 07 and T4 hand the member back to the app "with email pre-filled". The reference never records the actual path or parameters.

Currently implemented: we accept /app/return, /return-to-app and /app on attorney-shield.com and www.attorney-shield.com, and read an email query parameter. All of it is confined to one file so a confirmed contract is a one-line change.

We need: the real path and parameter names.

Security note, deliberate on our side: we treat the link as untrusted input. The email is a text-field prefill only - a link can never sign anyone in or convey entitlement, and we have a test asserting that a link carrying `accessToken`, `userID` or `roles` yields nothing but the email. Please do not design the link to carry a credential; if the app needs to know what someone bought, we would rather ask the backend after a real sign-in.

2.4 Domain verification files - HIGH (web task, not backend)

For the deep link to open the app *silently*, both hosts need:

- **Android:** `/.well-known/assetlinks.json` with our signing-certificate SHA-256 fingerprint for `com.app.attorney.shield`
- **iOS:** `/.well-known/apple-app-site-association` for our Team ID + `com.app.attorney.shield`

Until these exist, Android shows a "which app?" dialog and iOS universal links do not fire at all. We will send the fingerprint and Team ID once the release keystore exists.

3. Endpoints we need for the rest of the app

Grouped by the journey they unblock, most valuable first. Shapes below are **proposals** - they follow the conventions already in the gateway (see §4) so they should feel familiar, but please push back where the data model disagrees.

3.1 Native registration - screens 08-12 - BLOCKING Phase 3

This is the highest-value group, because it is the only thing that completes a journey. Today the flow is: web checkout -> deep link -> **dead end**. The app lands the member on sign-in because there is nowhere else to send them.

Five steps, in order:

Step	Screen	Collected
1	08	Mobile number (with +1 country code)
2	09	6-digit SMS code
3	10	Date of birth, gender, pronouns
4	11	Street, city, ZIP, "mailing address is the same"
5	12	4-digit security PIN

Proposed operations:

```
mutation SendPhoneVerification($input: SendPhoneVerificationInput!) {  
  sendPhoneVerification(input: $input) { sent expiresAt }
```

```

}
# input: { userID: ID!, phone: String! }

mutation VerifyPhone($input: VerifyPhoneInput!) {
  verifyPhone(input: $input) { verified }
}
# input: { userID: ID!, phone: String!, code: String! }

mutation UpdateMemberProfile($input: UpdateMemberProfileInput!) {
  updateMemberProfile(input: $input) { userID }
}
# input: { userID: ID!, dateOfBirth: String!, gender: String, pronouns: String }

mutation UpdateMemberAddress($input: UpdateMemberAddressInput!) {
  updateMemberAddress(input: $input) { userID }
}
# input: { userID: ID!, street: String!, city: String!, state: String,
# zip: String!, mailingSameAsPhysical: Boolean! }

mutation SetSessionPin($input: SetSessionPinInput!) {
  setSessionPin(input: $input) { userID }
}
# input: { userID: ID!, pin: String! }

query RegistrationStatus($userID: ID!) {
  registrationStatus(userID: $userID) {
    phoneVerified profileComplete addressComplete pinSet
  }
}
}

```

Three things worth confirming:

- 1 **Does completing registration return a session?** By step 5 the member has paid and proved ownership of a phone, so the backend has what it needs to issue one. If `setSessionPin` returns `accessToken/refreshToken`, the journey completes cleanly. If not, we drop them on sign-in, which is a poor ending to a paid signup.
- 2 `registrationStatus` **matters more than it looks.** People abandon signup and come back, or reinstall. Without it we cannot resume mid-flow and would restart them from step 1.
- 3 **The PIN is not an app password.** The reference is explicit: its only job is ending a live session securely and preventing accidental disconnection. It does not unlock the app or protect recordings. It should be verified server-side when a call is ended, not just held on the device.

3.2 Situation preferences - screens 13B, 13C, 27B - HIGH

Members pick up to three incident types to keep one tap from home. Small, and it visibly completes the home screen, which currently shows the full list because it has nowhere to store a choice.

```

query MemberSituations($userID: ID!) {
  memberSituations(userID: $userID) { incidentTypeID sortOrder }
}

mutation SetMemberSituations($input: SetMemberSituationsInput!) {
  setMemberSituations(input: $input) { incidentTypeID sortOrder }
}

# input: { userID: ID!, incidentTypeIDs: [ID!]! } # max 3

```

3.3 Document vault / "Digital Glovebox" - screens 14, 14A-14D, 30A, 31 - HIGH

Four categories in the reference: Driver's Information, Health Information, Gun Information, Citizenship Info. Visible to the attorney during a call (screen 30A) and access ends with the call.

Needs upload, list, view and delete. **We would prefer pre-signed upload URLs** over posting file bytes through the gateway.

```

query MemberDocuments($userID: ID!) {
  memberDocuments(userID: $userID) {
    id category fileName contentType sizeBytes uploadedAt
  }
}

mutation CreateDocumentUpload($input: CreateDocumentUploadInput!) {
  createDocumentUpload(input: $input) { documentID uploadUrl expiresAt }
}

# input: { userID: ID!, category: DocumentCategory!, fileName: String!,
# contentType: String!, sizeBytes: Int! }

mutation DeleteMemberDocument($documentID: ID!) { deleteMemberDocument(documentID: $documentID)
{ deleted } }

```

Questions: what are the size and MIME limits, how long are view URLs valid, and what exactly does "access ends when the call ends" mean on your side?

3.4 Activity timeline - screen 32 - LATER

```

query MemberActivity($userID: ID!, $limit: Int, $offset: Int) {
  memberActivity(userID: $userID, limit: $limit, offset: $offset) {
    id type occurredAt title subtitle callID
  }
}

```

Question: is this derived from call records you already have, or a new store?

3.5 Plan, payment method, family sub-accounts - screens 33A, 33B, 33D - LATER

Read-mostly. The reference is explicit that there are **no in-app plan changes**, and that delete-account lives only on 33A with App Store's required wording.

```
query MemberPlan($userID: ID!) {
  memberPlan(userID: $userID) {
    planName priceLabel renewalDate renewsEvery status # active | grace | expired
    paymentMethod { brand last4 expiryLabel }
    seats { used total }
  }
}

query FamilyMembers($userID: ID!) {
  familyMembers(userID: $userID) { id displayName email status }
}

mutation InviteFamilyMember($input: InviteFamilyMemberInput!) { ... }
mutation RemoveFamilyMember($input: RemoveFamilyMemberInput!) { ... }
mutation RequestAccountDeletion($userID: ID!) { requestAccountDeletion(userID: $userID) {
  requested } }
```

! We cannot build the family seat stepper yet. The design reference states capacity four different ways - "covers up to 5", "includes 3, add up to 2 more", "You + 3", "You + up to 4", "3 of 5 on your plan". Whether 5 includes the primary account is genuinely unclear. **Please confirm the real number and whether it is inclusive.**

3.6 Notifications and the nudge system - screens 15, 22-26 - LATER

A bell with unread count, a notification centre, gentle bottom-sheet nudges, and per-category settings.

```
query MemberNotifications($userID: ID!, $limit: Int) {
  memberNotifications(userID: $userID, limit: $limit) {
    id kind title body createdAt readAt actionDeepLink
  }
}

mutation MarkNotificationsRead($input: MarkNotificationsReadInput!) { ... }

query NotificationSettings($userID: ID!) { notificationSettings(userID: $userID) { category
  enabled } }

mutation UpdateNotificationSettings($input: UpdateNotificationSettingsInput!) { ... }
```

Question: is push (APNs/FCM) in scope, or is this in-app only for now? Push needs device-token registration and a sending service, which is a different size of task.

3.7 Trial and guest - screens V1-V2, T1-T8, G1-G3 - LATER

Lowest priority for us, and the one most entangled with \$2.5 below.

- **Trial:** a 7-day limited trial with a card on file, an in-app gate when a trial member taps to connect, and conversion to a paid plan.
- **Guest:** an unrecognised email at sign-in offers "continue as guest" with first/last name; guests browse the real home and hit a gate on every member feature.

We need to know whether a guest is a real account with a role, or a purely local state. That single answer changes the whole design.

4. Conventions we have matched

New endpoints that follow these will drop straight in.

- **GraphQL gateway:** POST {gatewayUrl}, Authorization: Bearer <accessToken> when authenticated. Errors surface as `errors[].message` and we show that text to the member, so please keep those messages human-readable - "invalid credentials" is genuinely more useful than a code.
- **Casing is inconsistent between services and we have mirrored it rather than tidied it:** the gateway uses `userId / organizationId / jurisdictionId`; the comms REST service uses `organizationId / memberUserId`. Worth knowing before you add a field on either side.
- **Comms REST:** POST {apiBaseUrl}/api/vonage/video/member-call, camelCase, 409 for no-attorney.
- **We never invent endpoints.** Where something does not exist, the app degrades visibly rather than faking it.

5. One security note

POST /api/vonage/video/member-call **takes no authentication today**. We are not designing around it and we have not built anything that depends on it staying open - but as it stands, anyone can place a call against any `organizationId/memberUserId` they can guess, and that call routes to a real attorney.

Flagging it rather than assuming it is known.

6. What we will do as each lands

You give us	We ship
Gateway URL + dev account	Real sign-in end to end; verified Home and Call against live data
An attorney online on dev	A proven video call, both platforms
Registration endpoints (§3.1)	Screens 08-12, completing the signup journey
Situations (§3.2)	The home screen's saved three, as designed

You give us	We ship
Deep-link contract (§2.3)	One-line change, then verified
Refresh answer (§2.1)	Either real refresh, or a documented decision to re-prompt

Anything in §3 we can start within a day of having the shapes - the screens are specified in the design reference and the app's architecture already separates network from UI.

Contact

Questions on any of the proposed shapes are welcome - they are a starting point shaped by the design reference and your existing conventions, not a demand. If the data model disagrees, tell us and we will follow it.